

The Interpretability Problem in Deep Reinforcement Learning: Credit Assignment in On-Policy and Off-Policy Algorithms

Rhian Kansara
rk955@bath.ac.uk

March 27, 2026

1 The Interpretability Problem in Deep RL

Deep reinforcement learning presents a unique problem within the AI space. Agents perform with superhuman precision and expertise in game environments such as Atari [Mnih et al., 2015] and Go [Silver et al., 2016] and even in robotics [Levine et al., 2016]. These feats can be attributed to the introduction of neural networks and deep learning — which enable more complex forms of function approximation — allowing for sophisticated strategies to emerge.

The cost associated with these new capabilities is interpretability: neural network function approximation and other deep learning methods (such as Deep Recurrent Reinforcement Learning) have outpaced our understanding for the most part. Deep reinforcement methods — while the most capable algorithms — are the least understood, and operate largely as a black box. By contrast, early methods of Reinforcement Learning (such as tabular methods and linear function approximation) offer complete transparency: troubleshooting policy collapse was as simple as inspecting value tables and weight vectors directly [Sutton and Barto, 2018].

This critique, while not new, becomes far more prevalent as practical applications of Reinforcement Learning are developed and deployed in the real world [Dulac-Arnold et al., 2021]. Understanding internal mechanisms would allow developers of deep RL algorithms to debug failures more effectively and ensure safety in the context of real-world deployment [Amodei et al., 2016].

This review seeks to examine the tools available to deconstruct these black boxes and applies them to the question: When an agent succeeds or fails, how does each algorithm family incorporate that outcome into its policy? We define success and failures at different levels. At the most basic level, a success is a transition in which the agent receives an above average immediate reward. At an episode level, this is achieving a higher return than average. These definitions however, imply that successful episodes are simply a series of positive transitions. This is hardly the case. An action may generate an above average return, but may sacrifice future rewards, while an action that has a negative return associated with it may enable future success. It's evident therefore that the raw

reward signal is simply not enough to evaluate an action’s utility. While value functions do attempt to capture long term consequences, their estimates are imperfect and evolve over training.

This review examines credit assignment in PPO and SAC, two dominant algorithms representing on-policy and off-policy learning, respectively. We analyse how each processes learning signals and survey interpretability methods — from visual saliency to gradient analysis and RSA — that can illuminate these mechanisms.

2 What is RL

In its simplest form, Reinforcement Learning (RL) is a computational approach to goal-directed learning through interaction [Sutton and Barto, 2018]. Unlike supervised learning, which relies on a dataset of labelled examples, an RL agent learns by taking actions in an environment and observing the consequences. Agents seek to maximise the cumulative reward over time, learning long-term strategies. The interaction is formalised as a Markov Decision Process, defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$: a set of states $\mathcal{S}$, a set of actions $\mathcal{A}$, a transition function $P(s'|s, a)$ describing environment dynamics, a reward function $R(s, a)$, and a discount factor $\gamma \in [0, 1]$ that weights immediate rewards against future ones [Sutton and Barto, 2018].

The agent’s goal is to learn a policy $\pi(a|s)$ (a mapping from states to actions) that maximises expected discounted return. Central to this is the concept of value: how good is it to be in a particular state? The state-value function $V^\pi(s)$ represents the expected return when starting from state s and following policy π :

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right] \quad (1)$$

Similarly, the action-value function $Q^\pi(s, a)$ represents expected return after taking action a in state s :

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right] \quad (2)$$

These value functions satisfy the Bellman Equation, a recursive relationship that forms the foundation of most RL algorithms:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V^\pi(s')] \quad (3)$$

Put simply, the bellman equations state that:

$$\text{Value of a state} = \text{Immediate Reward} + \text{Value of Successor States} \quad (4)$$

This recursive structure enables iterative algorithms that propagate reward information backwards through time the mechanism by which agents learn that early actions can lead to later rewards.

2.1 Model-Free vs. Model-Based Learning

One distinction among RL algorithms concerns how they use experience. Model-based methods learn an internal model of the environment’s dynamics and use it to plan or simulate experience [Hafner et al., 2020, Schrittwieser et al., 2020]. While sample-efficient, these methods introduce additional complexity: credit assignment depends on the accuracy of the learned model, and errors compound over planning horizons.

Model-free methods, by contrast, learn policies or value functions directly from experience without modelling transition dynamics. This simplicity comes at the cost of sample efficiency but avoids the compounding errors of imperfect world models. This review focuses on model-free methods, where credit assignment operates directly on experienced transitions rather than simulated ones.

2.2 On-Policy vs. Off-Policy Learning

A fundamental distinction in reinforcement learning algorithms concerns the relationship between the policy generating experience and the policy undergoing optimisation Dulac-Arnold et al. [2021]. We define the behaviour policy, β , as the distribution used to select actions during data collection; conversely, the target policy, π , is the distribution being learned. On-policy methods impose a strict equivalence requirement ($\beta = \pi$): the agent learns exclusively from data generated by its current behavioural distribution, rendering previous experience invalid after each parameter update. Off-policy methods remove this restriction ($\beta \neq \pi$); this allows the agent to learn retrospectively from data collected by earlier or entirely different policies.

A foundational approach to off-policy learning is Q-learning: a value-based mechanism that derives a policy implicitly by estimating the optimal action-value function rather than parameterising an actor directly. Because the algorithm updates its estimates by evaluating the highest-valued actions available in the subsequent state regardless of the action actually executed, it can decouple data collection from policy optimisation. Mnih et al. scaled this approach to high-dimensional state spaces by approximating the Q-function with a convolutional neural network: the Deep Q-Network (DQN) Mnih et al. [2015].

However, combining nonlinear function approximation with off-policy learning and bootstrapping creates severe instabilities; Sutton and Barto define this combination as the ‘deadly triad’ Sutton and Barto [2018]. In deep Q-learning, computing the temporal difference target requires a maximisation operator over the network’s own current estimates. Because neural network approximations inherently contain statistical noise, this operator selectively propagates positive noise upward: a systematic overestimation bias emerges, frequently precipitating catastrophic policy collapse.

To stabilise this process, DQN introduced two structural mechanisms. First, it implemented an experience replay buffer to store past transitions. Sampling randomly from this buffer breaks the temporal correlations inherent in sequential environment interactions; it provides a stationary data distribution for gradient descent. Second, it introduced a separate, periodically updated target network to compute Bellman backups. This architectural separation breaks the rapid feedback loops that cause overestimation to compound uncontrollably.

These algorithmic distinctions have direct implications for credit assignment. On-policy

algorithms process experience immediately: they assign credit while the context of that experience still reflects current algorithmic behaviour. Off-policy algorithms, facilitated by replay buffers and target networks, revisit stored transitions under updated value estimates; they retrospectively reinterpret the significance of past actions. Sections 3 and 4 examine representative algorithms from each family: PPO for on-policy learning and Rainbow for off-policy learning.

3 PPO and On-Policy Learning

The basis of model-free on-policy algorithms (or policy optimisation algorithms) is that the gradient can be expressed as an expectation over the policy’s own distribution, defined by the equation:

$$\nabla_{\theta} p(x) = p(x) \nabla_{\theta} \log p(x) \quad (5)$$

By using the log-derivative trick [Williams, 1992], policy gradient methods can remain agnostic to the underlying transition dynamics of the environment. While this affords significant generality, it creates an interpretability barrier: the resulting policy offers no direct access to the causal links between actions and outcomes that shaped it.

However, vanilla policy gradients suffer from high variance and instability—small changes to the policy can cause catastrophic performance collapse. Trust Region Policy Optimisation (TRPO) addressed this by constraining updates to stay within a “trust region” defined by KL-divergence from the previous policy [Schulman et al., 2015], but its second-order optimisation is computationally expensive.

Proximal Policy Optimisation (PPO) achieves similar stability through a simpler mechanism: a clipped surrogate objective [Schulman et al., 2017]. Rather than imposing a hard constraint, PPO clips the probability ratio between new and old policies, preventing excessively large updates:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (6)$$

where $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio and $\hat{A}_t$ is the estimated advantage. The clipping threshold ϵ (typically 0.2) ensures the policy changes gradually. When the ratio strays outside $[1 - \epsilon, 1 + \epsilon]$, the objective flattens, removing the incentive for further movement in that direction.

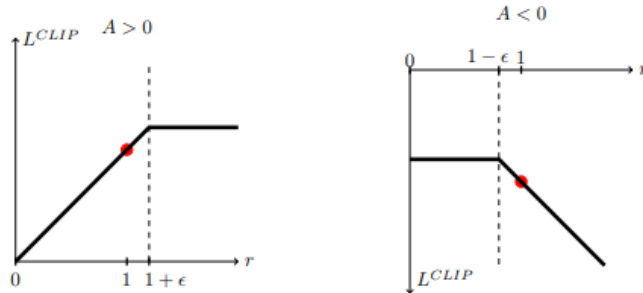


Figure 1: The PPO clipped objective. The red line shows the unclipped objective $r_t(\theta) \hat{A}_t$; the blue line shows the clipped version. When the probability ratio exceeds the clipping bounds, the gradient is zeroed, preventing destructive policy updates.

This design makes PPO both stable and practical, establishing it as a dominant algorithm for continuous control and game-playing tasks. However, the aggregation of many transitions into a single update obscures the contribution of individual experiences. Credit is assigned through advantages, but tracing whether a policy change was driven primarily by successful trajectories, failures, or some mixture is not straightforward.

3.1 Generalised Advantage Estimation and Temporal Methods

Generalised Advantage Estimation (GAE, Schulman et al. 2016) deals with credit assignment with a parametric weighting scheme over temporal distance. Rather than treating all rewards equally, as in Monte Carlo returns, or relying on single step estimation with TD(0), GAE computes advantage as an exponentially weighted average:

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \text{where } \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (7)$$

The hyperparameter $\lambda \in [0, 1]$ controls the decay rate (typically, λ is set between 0.95 and 0.99), which determines how credit is distributed to preceding actions. When $\lambda = 1$, GAE is essentially a Monte Carlo estimation, allowing credit to propagate across the full trajectory for the reward. When $\lambda = 0$, it becomes a single-step TD, and assigns credit to immediately preceding actions.

GAE balances Monte Carlo and single-step TD, weighting TD errors by temporal distance. This avoids the variance of full returns and the bias of single-step estimates. However, the weighting is a fixed heuristic: recent actions receive more credit than distant ones regardless of actual causal relevance. The hyperparameter λ assumes credit should decay as time progresses, an assumption that may not hold true in every environment. Consider Crafter [Hafner, 2021]: obtaining iron ingots requires first collecting ore, then smelting it. Under GAE’s temporal weighting, the earlier action (collecting ore) receives less credit than the later action (smelting), despite both being causally necessary for the outcome.

Further, GAE relies on leveraging a learned value function, inheriting the bias of that approximation. As the value function updates during training, the credit assignment itself becomes a moving target. Consequently, GAE does not distinguish which specific actions were responsible for observed outcomes: credit is assigned through temporal proximity rather than causal relevance.

Extensions to GAE, such as Adaptive GAE [Mohammadpour et al., 2024], attempt to refine the temporal heuristic by dynamically adjusting λ based on value-function error or explained variance. However, these variations merely calibrate the rate of decay for statistical stability: they do not alter the assumption that credit is a function of time rather than causal influence. Whether λ is fixed or adaptive, the mechanism remains largely the same, thereby inheriting the same problems of its predecessor while also introducing a 3rd moving target to the problem.

Despite these limitations, GAE remains the standard credit assignment mechanism in production PPO implementations [Schulman et al., 2017, Raffin et al., 2021]. As deep RL systems move from research benchmarks to real-world deployment [Dulac-Arnold et al., 2021], interpretability research must examine algorithms as they are actually used. Understanding how deployed agents process learning signals requires studying PPO with GAE, acknowledging its temporal heuristics as part of the system’s actual behaviour.

3.2 Learned Credit Assignment

To address the limitations of fixed temporal decay, methods such as Direct Advantage Estimation (DAE) [Pan et al., 2022] and RUDDER [Arjona-Medina et al., 2019] attempt to learn the credit assignment function from data rather than imposing it through a fixed heuristic. DAE reframes advantage estimation as a supervised regression problem, training a network to distribute observed returns across action sequences. RUDDER takes a different approach, using an LSTM to predict episode returns and redistributing credit based on when predictions change, effectively identifying “key events” that shift expected outcomes.

While conceptually appealing, both methods suffer from compounding instabilities. The learned credit assignment depends on a policy that is itself evolving during training, creating a moving target: the function must constantly adapt to explain a changing system. Furthermore, in attempting to interpret one black box (the policy network), these methods introduce a second learned black box (the advantage predictor or LSTM) whose internal reasoning is equally opaque. The result is that credit assignment becomes doubly removed from direct inspection — neither the policy’s behaviour nor the mechanism attributing credit to actions can be straightforwardly examined.

These methods demonstrate that temporal credit assignment is a recognised limitation in on-policy learning. However, replacing one learned function with another does not improve interpretability. Understanding how credit flows in deployed algorithms requires direct analysis of existing mechanisms, not their replacement.

4 Off-Policy Learning

Off-policy methods store transitions (s, a, r, s') in an experience replay buffer and sample from this memory to update value estimates. This enables significant sample efficiency: a single experience can be used for multiple learning updates, and the agent can learn from data collected by earlier, suboptimal versions of itself.

However, this flexibility introduces instability. The combination of function approximation (deep networks), bootstrapping (updating estimates based on other estimates), and off-policy data creates conditions prone to value divergence — what Sutton and Barto [2018] term the “deadly triad.” Because training data comes from a different distribution than the current policy, the agent is susceptible to extrapolation error, systematically overestimating the value of actions it has not recently attempted.

From an interpretability standpoint, off-policy learning presents a distinct challenge. Credit assignment occurs retrospectively: the agent revisits stored experiences and reinterprets them under its current value estimates. A transition that was uninformative early in training may become highly influential later, or vice versa. Unlike on-policy methods, where the learning signal flows immediately from action to update, off-policy algorithms maintain a complex, evolving relationship between experience and credit — which is primarily balanced by the replay buffer’s sampling distribution and the changing value function. Understanding how these algorithms differentially weight successes and failures requires examining not just what they learn, but how they selectively revisit and reinterpret past experience.

4.1 Rainbow and Distributional Q-Learning

As established in Section 2.2, vanilla Deep Q-Networks (DQN) suffer from severe overestimation bias and delayed reward propagation. While the introduction of uniform experience replay and periodic target networks prevented immediate policy collapse, these mechanisms failed to resolve the underlying vulnerabilities of the deadly triad. Rainbow Hessel et al. [2017] addresses these persistent instabilities by integrating several independent extensions into a single comprehensive architecture.

To counter the overestimation bias caused by DQN’s scalar maximisation operator, Rainbow introduces distributional reinforcement learning Bellemare et al. [2017]. Rather than estimating the expected scalar return $Q(s, a) = \mathbb{E}[Z(s, a)]$, Rainbow models the full probability distribution of the random return $Z(s, a)$. This distribution is parameterised over a discrete set of supports, defined as “atoms” z_i , with corresponding probabilities $p_i(s, a)$. Consequently, parameter updates are driven by minimising the Kullback-Leibler (KL) divergence between the predicted distribution and the projected Bellman target. While this distributional perspective mitigates the instability of off-policy bootstrapping, it creates a distinct interpretability barrier: learning signals manifest as shifts in probability mass rather than direct reward magnitudes.

To address DQN’s notoriously slow reward propagation, Rainbow replaces single-step updates with multi-step temporal difference learning. Instead of bootstrapping from the immediately subsequent state, the target is extended to an n -step return:

$$R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n \max_{a'} Q(s_{t+n}, a') \quad (8)$$

This mathematical formulation acts as a fixed temporal heuristic that bridges short causal horizons; however, it imposes a rigid assumption on how credit propagates backward through the trajectory.

Crucially, Rainbow abandons DQN’s uniform sampling of the replay buffer in favour of Prioritized Experience Replay (PER) Schaul et al. [2016]. Under uniform sampling, every stored transition i in a buffer of capacity N has an equal probability of selection: $P(i) = \frac{1}{N}$. PER fundamentally alters this data distribution by weighting the sampling probability according to the transition’s temporal difference error magnitude, δ_i . The prioritized probability is defined as:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \quad (9)$$

where $p_i = |\delta_i| + \epsilon$ represents the priority of the transition, and α dictates the degree of prioritisation. To correct the resulting bias in the gradient expectation, PER applies importance sampling weights during the update: $w_i = \left(\frac{1}{N \cdot P(i)}\right)^\beta$. While this targeted sampling improves sample efficiency, it introduces an implicit form of credit assignment. Because experiences the model computes as mathematically “surprising” receive disproportionate influence on the parameter updates, understanding how deployed agents weight success and failure must account for this dynamically shifting sampling distribution; uniform sampling assumptions are no longer valid.

Despite this stability, Rainbow remains inherently model-free with implicit credit assignment. Because reward signals propagate backward through iterated distributional Bellman updates, the algorithm correlates states with values without explicitly modelling causality; over long horizons, this mathematical link degrades across bootstrap steps, obscuring which specific experiences actually drive policy improvement.

4.2 Counterfactual Credit Assignment

To address implicit credit assignment in off-policy learning, Hindsight Credit Assignment (HCA) reframes the problem through counterfactual reasoning [Harutyunyan et al., 2019]. Rather than waiting for values to converge through Bellman backups, HCA quantifies the causal impact of an action as a likelihood ratio:

$$\text{Credit}(s, a, g) = \frac{P(g \mid s, a)}{P(g \mid s)} \quad (10)$$

If the ratio exceeds 1, the action caused the goal more than average behaviour; if less than 1, it hindered it.

However, this formulation requires sufficient successful transitions to model $P(g \mid s, a)$. In sparse-reward environments, goal states are rare, and supervised models trained on imbalanced data collapse to near-zero predictions. Hindsight Experience Replay (HER) [Andrychowicz et al., 2018] addresses this by relabelling failed trajectories with goals that were actually achieved, treating every episode as a success for some outcome.

The combination creates a compounded interpretability problem. HCA trained on relabelled data learns causality from hallucinated tasks that diverge from the true objective. The credit signal quantifies contribution toward incidental states, not task goals. Furthermore, the HCA estimator faces a moving target: it must learn ratios based on an evolving policy (changing the denominator) and an artificially shifting goal distribution (changing the numerator). Credit assignment becomes doubly detached: derived from a changing policy on synthetic data.

HCA shows that explicit credit assignment in off-policy learning remains an open problem. However, HCA has not been adopted in standard SAC implementations [Raffin et al., 2021], remaining a research proposal rather than deployed practice. Understanding how production algorithms process success and failure requires analysing SAC as it is actually used -with implicit credit assignment via Bellman backups rather than experimental extensions.

5 Interpretability Methods

The preceding sections have established that both PPO and SAC assign credit through mechanisms that are not directly inspectable: temporal weighting and advantage aggregation for PPO, iterated Bellman updates for SAC. Neither algorithm exposes which experiences drive policy changes. This section surveys methods that could illuminate these processes from the outside — techniques for examining what trained policies attend to, how individual inputs shape learning, and what representational structures emerge.

5.1 Static Policy Analysis

Static interpretability methods analyse trained policies to understand what information drives decisions. These approaches treat the policy as fixed and ask: given this learned behaviour, what features or inputs does the agent attend to?

Greydanus et al. [2018] introduced perturbation-based saliency for Atari agents. By applying a localised Gaussian blur to regions of the input and measuring the resulting policy divergence, this method identifies which pixels influence the agent’s decisions. Figure 2 illustrates this approach: highlighted regions indicate where perturbations most affect action selection, revealing whether an agent responds to task-relevant features or spurious correlations.

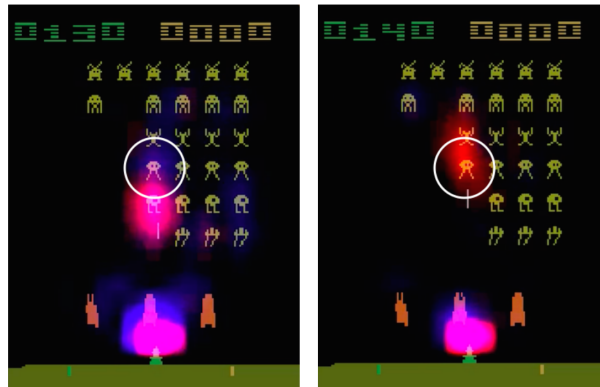


Figure 2: Perturbation-based saliency maps for an Atari agent. Highlighted regions indicate pixels whose modification most affects the policy output. From Greydanus et al. [2018].

Beyond visual inspection, Beechey et al. [2023] apply Shapley values to reinforcement learning through SVERL (Shapley Values for Explaining Reinforcement Learning). This game-theoretic approach quantifies the marginal contribution of each state feature to the agent’s expected return, providing a principled decomposition of performance. Figure 3 shows an example attribution. However, exact Shapley computation requires evaluating 2^n feature subsets, making it intractable for high-dimensional inputs. Approximation methods introduce variance, and the computational cost renders longitudinal analysis across training infeasible.

These methods have the same limitation: they explain the product of learning, not the process. Saliency maps and Shapley attributions describe what a trained agent has learned to attend to, but offer no insight into how learning signals from success and failure shaped that attention over training.

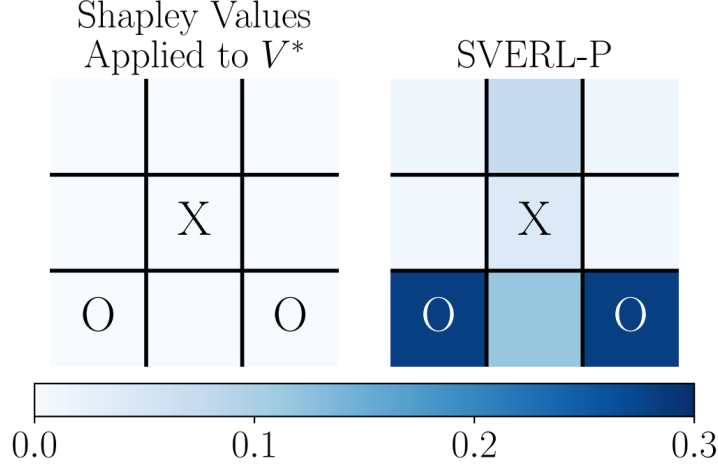


Figure 3: SVERL feature attribution for a trained agent. Bar heights indicate each feature’s Shapley value contribution to expected return. From Beechey et al. [2023].

5.2 Gradient-Based Analysis

Gradient-based methods examine learning signals directly — examining how experience shapes parameter updates.

The gradient $\nabla_{\theta}L(z, \theta)$ describes the direction in parameter space that would reduce loss on input z . Comparing gradients across inputs reveals relationships between learning signals. Szolnoky et al. [2022] propose Model Gradient Similarity (MGS), using cosine similarity to measure whether two inputs push the network in similar directions:

$$\text{sim}(z_1, z_2) = \frac{\nabla_{\theta}L(z_1, \theta) \cdot \nabla_{\theta}L(z_2, \theta)}{\|\nabla_{\theta}L(z_1, \theta)\| \|\nabla_{\theta}L(z_2, \theta)\|} \quad (11)$$

High similarity indicates reinforcing updates; negative similarity indicates conflicting signals.

For attributing model behaviour to specific training data, Koh and Liang [2017] introduced influence functions to deep learning. Originating in robust statistics, influence functions estimate how upweighting or removing a training point would affect model parameters without retraining:

$$\mathcal{I}(z) = -H_{\theta}^{-1} \nabla_{\theta}L(z, \theta) \quad (12)$$

where H_{θ} is the Hessian of the loss. This traces predictions back to the training examples that most shaped them.

Both approaches extend naturally to reinforcement learning. For on-policy algorithms, trajectories can be partitioned by episode outcome, with gradients computed separately on successful and failed experiences. For off-policy algorithms, the replay buffer enables retrospective partitioning: transitions can be tagged by the outcome of their originating episode, allowing gradient analysis on success-only or failure-only subsets. This reveals whether the algorithm receives differentiated learning signals from each outcome type, whether success and failure push the policy in opposing directions, and with what relative magnitude.

Influence functions provide a more rigorous framework for this analysis, quantifying not just gradient direction but the estimated impact on model parameters. However, their reliance on Hessian computation introduces scalability challenges for large networks.

5.3 Representational Similarity Analysis

Representational Similarity Analysis (RSA) examines the internal structure of neural network representations. Originally developed in neuroscience to compare brain activity patterns [Kriegeskorte et al., 2008], RSA constructs a representational dissimilarity matrix (RDM) by computing pairwise distances between activations for a set of inputs:

$$\text{RDM}_{ij} = 1 - \text{sim}(h(x_i), h(x_j)) \quad (13)$$

where $h(x_i)$ is the activation for input x_i . The RDM captures which inputs the network treats as similar or distinct.

RSA enables comparisons that would otherwise be intractable. Two networks with different architectures cannot be directly compared at the parameter level, but their RDMs can. If both produce similar RDMs, they have learned similar representational structures regardless of implementation [Mehrer et al., 2020].

Applied to reinforcement learning, RSA can track how representations evolve during training. By computing RDMs at successive checkpoints for a fixed set of states, one can observe whether the agent develops structured representations of task-relevant features. Where gradient analysis reveals learning signal direction, RSA reveals the resulting representational geometry.

6 Summary

The interpretability methods reviewed — visual saliency, gradient analysis, influence functions, and RSA — have each been applied to neural networks in isolation. Saliency maps explain what trained agents attend to. Gradient analysis reveals how individual inputs shape parameter updates. RSA characterises representational geometry. Yet no work has combined these approaches to study how different reinforcement learning algorithms process experience.

A particular gap exists in comparing algorithm families. On-policy and off-policy methods differ fundamentally in how they assign credit: on-policy methods require $\beta = \pi$, learning only from data generated by the current policy, while off-policy methods permit $\beta \neq \pi$, assigning credit under the current policy to actions taken by earlier or different policies. These differences should manifest in gradient signatures and representational structure, but no systematic comparison exists. Recent work has begun applying mechanistic interpretability to RL agents. Trim and Grayston [2024] use saliency mapping and feature visualisation to study goal misgeneralisation in a PPO-trained maze agent. However, such studies examine individual algorithms rather than comparing how different algorithm families process experience.

Furthermore, existing work focuses on trained policies rather than the learning process. Static analysis reveals what an agent has learned, but not how it learned from success versus failure. Existing applications of gradient analysis and RSA to neural networks

have been cross-sectional rather than longitudinal — tracking these properties across training checkpoints would reveal learning dynamics, but this extension has not been systematically pursued in reinforcement learning.

Bridging these gaps — combining complementary methods, comparing across algorithm families, and examining learning dynamics rather than final performance — would reveal how deployed algorithms differentially process success and failure.

The methods reviewed in Section 5 have been developed and validated largely in isolation. Saliency and Shapley approaches explain what trained policies attend to, but not how those attention patterns emerged during learning. Gradient-based methods can track learning dynamics but have not been systematically applied across algorithm families or training stages. RSA enables architecture-agnostic comparison but remains underutilised in RL. No existing work combines these approaches to compare how on-policy and off-policy algorithms differentially process successful versus failed experience across the course of training.

This pattern has been documented in research settings: Trim and Grayston [2024] show a maze-solving agent that learned to navigate toward the top-right corner rather than toward the goal, ignoring cheese placed elsewhere. In a benchmark, this is a curiosity. In a warehouse robot or surgical assistant learning from experience, the same failure mode — learning spurious correlations rather than causal task structure — could cause harm before existing evaluation metrics reveal the problem.

Current evaluation practices assess what policies do, not why they do it. Bridging the interpretability methods reviewed here with systematic comparison of credit assignment across algorithm families would provide tools for auditing learning dynamics before deployment, rather than diagnosing failures after them.

References

- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in ai safety. *arXiv preprint arXiv:1606.06565*, 2016.
- Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. *arXiv preprint arXiv:1707.01495*, 2018.
- Jose A Arjona-Medina, Michael Gillhofer, Michael Widrich, Thomas Unterthiner, Johannes Brandstetter, and Sepp Hochreiter. Rudder: Return decomposition for delayed rewards. *Advances in Neural Information Processing Systems*, 32, 2019.
- Daniel Beechey, Thomas M S Smith, and Özgür Şimşek. Explaining reinforcement learning with Shapley values. *Proceedings of the 40th International Conference on Machine Learning*, pages 2003–2014, 2023.
- Marc G Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. *arXiv preprint arXiv:1707.06887*, 2017.
- Gabriel Dulac-Arnold, Nir Levine, Daniel J Mankowitz, Jerry Li, Cosmin Paduraru, Sven Gowal, and Todd Hester. Challenges of real-world reinforcement learning: definitions, benchmarks and analysis. *Machine Learning*, 110(9):2419–2468, 2021.
- Sam Greydanus, Anurag Koul, Jonathan Dodge, and Alan Fern. Visualizing and understanding Atari agents. *Proceedings of the 35th International Conference on Machine Learning*, pages 1792–1801, 2018.
- Danijar Hafner. Crafter. *Github*, 2021.
- Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *International Conference on Learning Representations*, 2020.
- Anna Harutyunyan, Will Dabney, Thomas Mesnard, Mohammad Gheshlaghi Azar, Bilal Piot, Nicolas Heess, Hado van Hasselt, Gregory Wayne, Satinder Singh, Doina Precup, and Remi Munos. Hindsight credit assignment. *Advances in Neural Information Processing Systems*, 32, 2019.
- Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. *arXiv preprint arXiv:1710.02298*, 2017.
- Pang Wei Koh and Percy Liang. Understanding black-box predictions via influence functions. *Proceedings of the 34th International Conference on Machine Learning*, 70: 1885–1894, 2017.
- Nikolaus Kriegeskorte, Marieke Mur, and Peter A Bandettini. Representational similarity analysis - connecting the branches of systems neuroscience. *Frontiers in Systems Neuroscience*, 2:4, 2008.

- Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research*, 17(1):1334–1373, 2016.
- Johannes Mehrer, Courtney J Spoerer, Emer C Jones, Nikolaus Kriegeskorte, and Tim C Kietzmann. Individual differences among deep neural network models. *Nature Communications*, 11(1):5725, 2020. doi: 10.1038/s41467-020-19632-w.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533, 2015.
- Naemeh Mohammadpour, Meysam Fozi, and Mohammad Mehdi Ebadzadeh. Proximal policy optimization with adaptive generalized advantage estimate: Critic-aware refinements. *Expert Systems with Applications*, 238:122321, 2024.
- Fei Pan, Jiaming He, Dandan Tu, and Qing He. Understanding and improving deep reinforcement learning via direct advantage estimation. *arXiv preprint arXiv:2109.06093*, 2022.
- Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL <http://jmlr.org/papers/v22/20-1364.html>.
- Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. *International Conference on Learning Representations*, 2016.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.
- John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. *Proceedings of the 32nd International Conference on Machine Learning*, pages 1889–1897, 2015.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *International Conference on Learning Representations*, 2016.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.

- Vincent Szolnoky, Tobias Andersson, Balazs Kulcsár, and Fredrik Gustafsson. On the interpretability of regularisation for neural networks through model gradient similarity. *Advances in Neural Information Processing Systems*, 35:26726–26739, 2022.
- Tristan Trim and Triston Grayston. Mechanistic interpretability of reinforcement learning agents. *arXiv preprint arXiv:2411.00867*, 2024.
- Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.